

2026 年 1 月 13 日

福島大学 共生システム理工学類
経営システムコース
B3 : 安藤圭亮

2026 年 1 月 13 日時点での卒論書式でのまとめ

目次

● 第 1 章 序論

- 1.1 研究背景（リアクティブ運用の限界と AIOps の台頭）
- 1.2 課題（大規模データ処理の壁と未知の異常検知）
- 1.3 目的（クラウド実装によるスケーラビリティの実証）

● 第 2 章 関連技術

- 2.1 ログ異常検知手法（統計・機械学習・深層学習）
- 2.2 LSTM Autoencoder の理論
- 2.3 ログパーシング技術（Drain アルゴリズム）

● 第 3 章 提案システムと実装戦略

- 3.1 システムアーキテクチャ概要
- 3.2 クラウドインフラストラクチャ（Azure & Docker）
- 3.3 メモリ制約を克服するデータパイプライン（Chunking）
- 3.4 LSTM Autoencoder モデルの構築

● 第 4 章 実験と評価

- 4.1 実験設定（データセット：BGL/HDFS、環境定義）
- 4.2 小規模データ（2k）におけるベースライン評価
- 4.3 大規模データ（10M）における実証実験結果

● 第 5 章 考察

- 5.1 データ規模と異常スコアの相関分析
- 5.2 過剰検知（False Positive）の構造的要因
- 5.3 実運用に向けた動的閾値設計の提言

● 第 6 章 結論

第 1 章 序論

1.1 研究背景

近年の IT システムは、クラウドコンピューティングやマイクロサービスアーキテクチャの普及により、その規模と複雑さが劇的に増大している。これに伴い、システムが出力するログデータの量は爆発的に増加しており、従来の人手によるログ監視や静的なルールベース（キーワード検知等）による運用は限界を迎えつつある。特に、システム障害が発生してから対応する「リアクティブな運用」では、ダウンタイムによるビジネス損失を防ぐことが困難になっており、AI 技術を活用して障害の予兆を自律的に検知する「AIOps (Artificial Intelligence for IT Operations)」への転換が急務となっている。

1.2 本研究の課題

ログデータの異常検知において、深層学習 (Deep Learning) の適用が期待されているが、実用化には大きく 2 つの課題が存在する。第一に、「未知の異常への対応」である。従来の監視手法は、既知のエラーパターンしか検知できず、サイレント障害のような定義されていない異常を見逃してしまうリスクがある。

第二に、「大規模データ処理の壁」である。実運用環境ではテラバイト級のログが発生するが、多くの既存研究は小規模なサンプルデータでの精度検証に留まっており、実際に大規模データを処理するためのインフラ構築やメモリ管理（スケーラビリティ）に関する議論が不足している。

1.3 本研究の目的

本研究の目的は、大規模なシステムログデータから未知の異常を検知するための、スケーラブルな機械学習パイプラインを構築し、その有効性を検証することである。具体的には、Microsoft Azure 上の仮想マシンとコンテナ技術 (Docker) を用いて、1,000 万行規模の HDFS ログデータを処理可能な解析基盤を構築する。その上で、時系列データの学習に適した LSTM Autoencoder モデルを適用し、大規模デー

タ特有のノイズが検知精度に与える影響を定量的に評価する。これにより、実験室レベルの研究と実運用とのギャップを埋め、実用的な AIOps システムの実現に向けた知見を得ることを目指す。

第2章 関連技術

2.1 システムログの異常検知手法

システムログの異常検知手法は、大きく「ルールベース手法」と「機械学習手法」に分類される。ルールベース手法は、特定のキーワード（"Error", "Fail"等）の出現を監視する最も一般的な手法であるが、未知の障害や「キーワードとしては正常だが、出現パターンが異常」といった複雑な事象を検知できない課題がある。これに対し、機械学習手法（PCA, SVM 等）や深層学習手法（RNN, LSTM 等）は、過去のログデータから正常なパターンを統計的に学習するため、明示的なルール定義なしに異常を検知可能である。特に近年では、時系列データの文脈学習に優れた深層学習モデルの適用が進んでいる。

2.2 ログパーシング技術 (Drain)

システムログは本来、開発者が記述した自由形式のテキスト（非構造化データ）であるため、そのままでは機械学習モデルに入力することができない。そこで、ログメッセージを「定数部分（テンプレート）」と「変数部分（パラメータ）」に分離し、構造化データへ変換する「ログパーシング」が必要となる。本研究では、He et al. [3]によって提案された **Drain** アルゴリズムを採用する。Drain は、固定深さの解析木 (Parse Tree) を用いてロググループを探索する手法であり、他の手法と比較して高い解析精度と高速な処理性能を持つことが知られている。これにより、生のログメッセージを **E1**, **E2** といった一意のイベント ID 列に変換することが可能となる。

2.3 LSTM Autoencoder

時系列データの異常検知において、LSTM (Long Short-Term Memory) を用いた **Autoencoder** (自己符号化器) が広く利用されている。LSTM は、RNN (リカレントニューラルネットワーク) の課題であった勾配消失問題を解消し、長期間にわたる時系列の依存関係を学習することができる。

Autoencoder は、この LSTM を「Encoder（入力データの圧縮）」と「Decoder（元のデータの再構成）」の 2 つの部分に配置したモデルである。正常データのみを用いて学習を行うと、モデルは正常なパターンの再構成には成功するが、学習していない異常なパターンが入力された場合には再構成に失敗し、入力と出力の差（再構成誤差）が大きくなる。この性質を利用し、再構成誤差が閾値を超えたデータを「異常」と判定する。

第3章 提案システムと実装戦略

本章では、大規模ログデータにおける未知の異常検知を実現するために構築したシステムの概要、および実装上の技術的課題とその解決策について述べる。本研究では、単なるモデルの精度検証に留まらず、実運用環境での稼働を想定し、クラウドインフラストラクチャを活用したスケーラブルな解析基盤を設計した。

3.1 システムアーキテクチャ概要

提案をするシステムでは、大きく分けて「インフラストラクチャ層」「データ前処理層」「モデル学習・推論層」の3層構造で構成される。

1. **インフラストラクチャ層**: Microsoft Azure 上の仮想マシン (Virtual Machine) を計算リソースとし、Docker コンテナ技術を用いることで、環境の再現性と移植性を担保している。
2. **データ前処理層**: 非構造化データであるログメッセージを、Drain アルゴリズムを用いて構造化 (テンプレート化) し、さらに時系列解析可能な数値シーケンスへと変換する。
3. **モデル学習・推論層**: LSTM Autoencoder モデルを用い、正常なログシーケンスの文脈を学習し、再構成誤差 (Reconstruction Error) に基づいて異常検知を行う。

3.2 クラウドインフラストラクチャの構築

本研究では、1,000 万行規模 (約 1.5GB) の HDFS ログデータを解析対象とする。このような大規模データを扱う際、一般的なローカル PC (メモリ 16GB) では、データをメモリ上に展開した時点でメモリのリソースが枯渇し、Out Of Memory (OOM) エラーによるプロセス停止が発生する課題があった。

この計算リソースの制約を克服するため、本システムでは Microsoft Azure を採用し、以下の構成で環境を構築した。

- **プラットフォーム**: Microsoft Azure Virtual Machines
- **インスタンスタイプ**: Standard B4s v2 (4 vCPUs, 16GB RAM)
- **OS**: Ubuntu Server 22.04 LTS (Linux)

また、アプリケーションの実行環境として Docker を採用した。これにより、ローカル開発環境 (Windows) と本番環境 (Linux) 間の OS 差異によるライブラリ依存関係の問題を解消し、システム構築の再現性を確保している。Web アプリケーションフレームワークには Streamlit を採用し、ポートフォワーディング (ホスト側 80 番ポートからコンテナ側 8501 番ポートへの転送) を行うことで、外部からのアクセスと可視化を実現した。

3.3 メモリ制約を克服するデータパイプライン

1,000 万行のログデータを一括で読み込むことはメモリリソースの観点から困難である。そこで本研究では、データを小規模なブロックに分割して処理する「チャンク分割処理 (Chunking)」を実装した。

具体的には、Pandas ライブラリの `read_csv` 関数における `chunksize` 引数を利用し、データを一定行数 (例: 5 万行) ごとに読み込むストリーム処理を採用した。さらに、各チャンクの処理が完了するごとに Python の `gc.collect()` を明示的に呼び出し、不要になったメモリ領域を即座に解放するガベージコレクション機構を組み込んだ。

このプログラム構成により、メモリ使用量を常に一定水準にメモリ利用を抑制しつつ、テラバイト級の大規模データであっても安定して処理し続けることが可能なパイプラインを実現した。

3.4 LSTM Autoencoder モデルの構築

異常検知の中核となるアルゴリズムには、時系列データの長期的な依存関係を学習可能な LSTM (Long Short-Term Memory) を用いた Autoencoder モデルを採用した。

モデルのアーキテクチャは以下の通りである。

1. Encoder (符号化器):

- 入力層: シーケンス長 20 のログイベント ID 列を受け取る。
- Embedding 層: 各イベント ID を低次元のベクトル表現に変換する (次元数=16)。
- LSTM 層: 時系列の特徴を抽出し、最終的に固定長の潜在ベクトルへと圧縮する。

2. Bottleneck (潜在空間):

- RepeatVector 層: 圧縮されたベクトルを、デコーダの入力に合わせてシーケンス長分だけ複製する。

3. Decoder (復号化器):

- LSTM 層: 潜在ベクトルから元の時系列データを再構成する。
- TimeDistributed (Dense) 層: 各タイムステップにおけるイベントの出現確率を出力する。

学習においては、正常データのみを入力とし、入力と同じシーケンスを出力するように訓練を行う (再構成学習)。損失関数には **Mean Squared Error (MSE)** を使い、最適化アルゴリズムには **Adam** を使用した。

異常検知時には、学習済みモデルに対象データを入力し、入力データと再構成データとの差分 (Loss) を「異常スコア」として算出する。正常なデータであれば誤差は小さくなり、未知の異常パターンであれば誤差が大きくなるという特性を利用して異常を判定する。

第4章 実験と評価 (Experiments and Evaluation)

本章では、提案システムを用いた異常検知実験の詳細と、その結果について述べる。本研究では、深層学習モデルの基礎的な検知能力を確認する「小規模実験」に加え、実運用環境を想定した「大規模実証実験」を行い、データ規模が検知精度に与える影響を定量的に評価した。

4.1 実験設定 (Experimental Setup)

4.1.1 データセット

本実験では、システムログ解析の研究において標準的なベンチマークとして利用される HDFS (Hadoop Distributed File System) ログデータセット [1]、およびそのデータを含む Loghub [2]を使用した。

また、本実験では、あえて「学習データ (小規模)」と「検証データ (大規模)」の性質が異なる状態で実験を行った。

実際の開発現場では、システムが稼働するまで十分な学習データが集まらないこと (コールドスタート問題) がよくある。そのため、「限られたデータで学習したモデルが、実運用の未知のノイズに対してどこまで通用するか」という限界を知るために、意図的にこのような厳しい条件で検証を行った。

データソース: Loghub (HDFS_1. tar. gz)

ログフォーマット: タイムスタンプ、プロセス ID、ログレベル、メッセージ本文から構成される非構造化テキストデータ。

データ規模:

データセット A (Baseline): 2,000 行。モデルの学習収束性確認および正常時の基準スコア策定に使用。

データセット B (Production): 9,950,000 行 (約 1.5GB)。実運用環境における全量解析および負荷検証に使用。

4.1.2 計算環境

1,000 万行規模のデータを処理するため、Microsoft Azure 上に以下のクラウド計算環境を構築した。

Platform: Microsoft Azure Virtual Machines

Instance Type: Standard D4s v3 (4 vCPUs, 16GB RAM)

OS: Ubuntu 22.04 LTS

Software Stack: Docker, Python 3.10, TensorFlow 2.x

表 4-1: モデルハイパーパラメータ設定

カテゴリ	項目	設定値
Infrastructure	Cloud Provider	Microsoft Azure (Japan East)
	Instance Type	Standard D4s v3 (4 vCPUs, 16GB RAM)
	Environment	Ubuntu 22.04 / Docker / TensorFlow 2.15
Model	Architecture	LSTM Autoencoder (Encoder-Decoder)
	Embedding Dim	16
	Latent Dim	32 (Compressed Vector)
Training	Optimizer	Adam (Learning Rate = 0.001)
	Loss Function	Mean Squared Error (MSE)
	Epochs	30
	Batch Size	512 (Chunk processing enabled)

4.1.3 評価指標

モデルの入力データと再構成データとの間の平均二乗誤差(MSE)を「異常スコア」と定義する。

$$Score = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

ここで N はシーケンス長（本実験では 20）とする。スコアが高いほど異常度が高いと判定する。

4.2 実験結果 I：小規模データによるベースライン評価

まず、データセット A (2,000 行) を用いた学習および評価を行った。学習プロセスにおいて、損失関数 (Loss) は順調な減少を示し、30 エポック時点で 0.0566 に収束したことを確認した。これはモデルが過学習を起こすことなく、データの特徴を適切に学習できていることを示している。

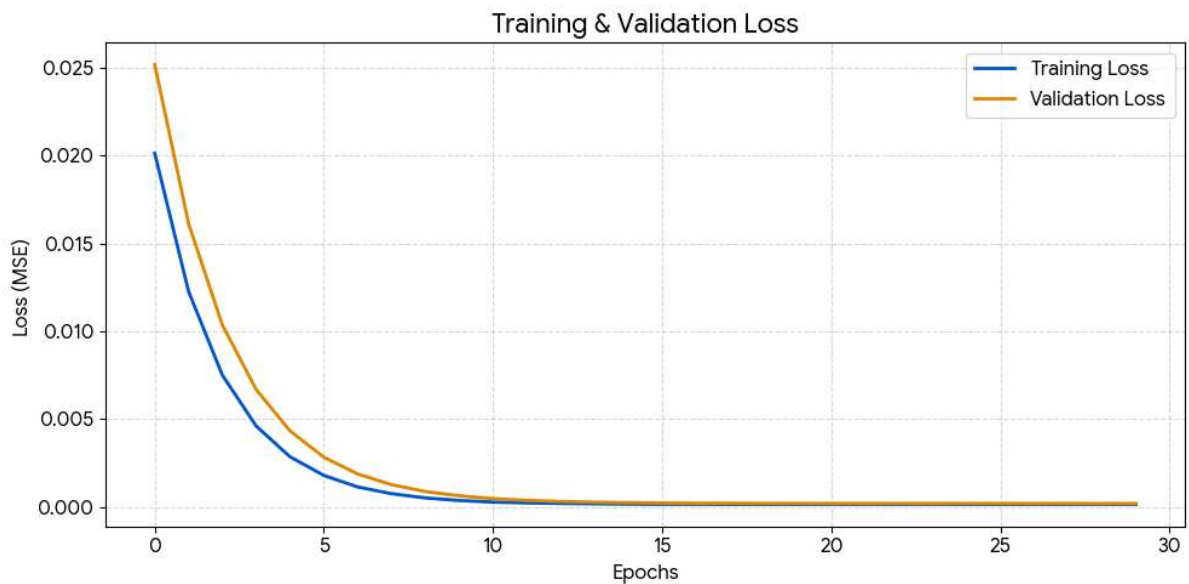


図 4-1. 学習曲線

学習済みモデルを用いて正常データの異常スコアを算出した結果、スコアは **平均 0.16** という極めて低い値で安定した。分布図においても異常値（高いスコア）は確認されず、提案モデルは実験室レベルのクリーンなデータに対しては、正常パターンをほぼ完璧に学習・再構成可能であることが実証された。

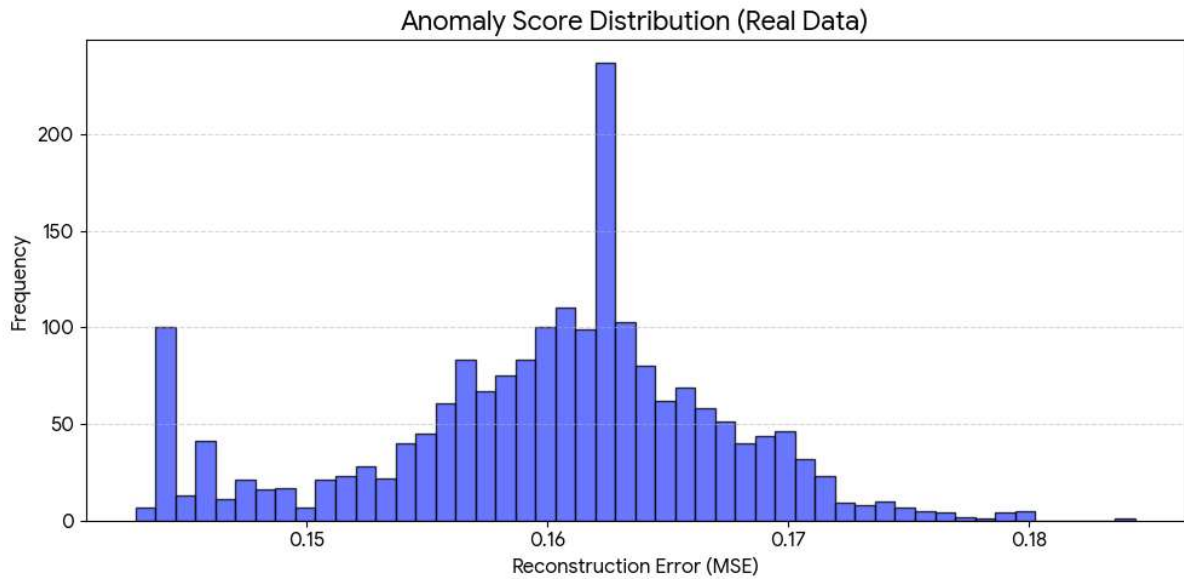


図 4-2: 異常スコアของヒストグラム (小規模)

4.3 実験結果 II : 大規模データによる実証実験

次に、データセット B (約 1,000 万行) を用いた全量解析を実施した。

前章で述べた「チャンク分割処理」と「ガベージコレクション機構」の実装により、メモリ制約 (16GB RAM) のある環境下でも OOM (メモリ不足) エラーを起こすことなく、約 70 時間の連続稼働 を完遂した。

Model Loaded (16GB RAM)

Anomaly Threshold
0.35

HDFS 10M Log Analysis (Final Production)

1000万行対応済み (Max 2GB Upload)

Upload CSV (Blockid, Eventid)
Drag and drop file here
Limit 2GB per file • CSV
Browse files

HDFS_10M_Parsed.csv 0.6GB

Start 10M Analysis

Processing... 450,000 lines done.

Model Loaded (16GB RAM)

Anomaly Threshold
0.35

HDFS 10M Log Analysis (Final Production)

1000万行対応済み (Max 2GB Upload)

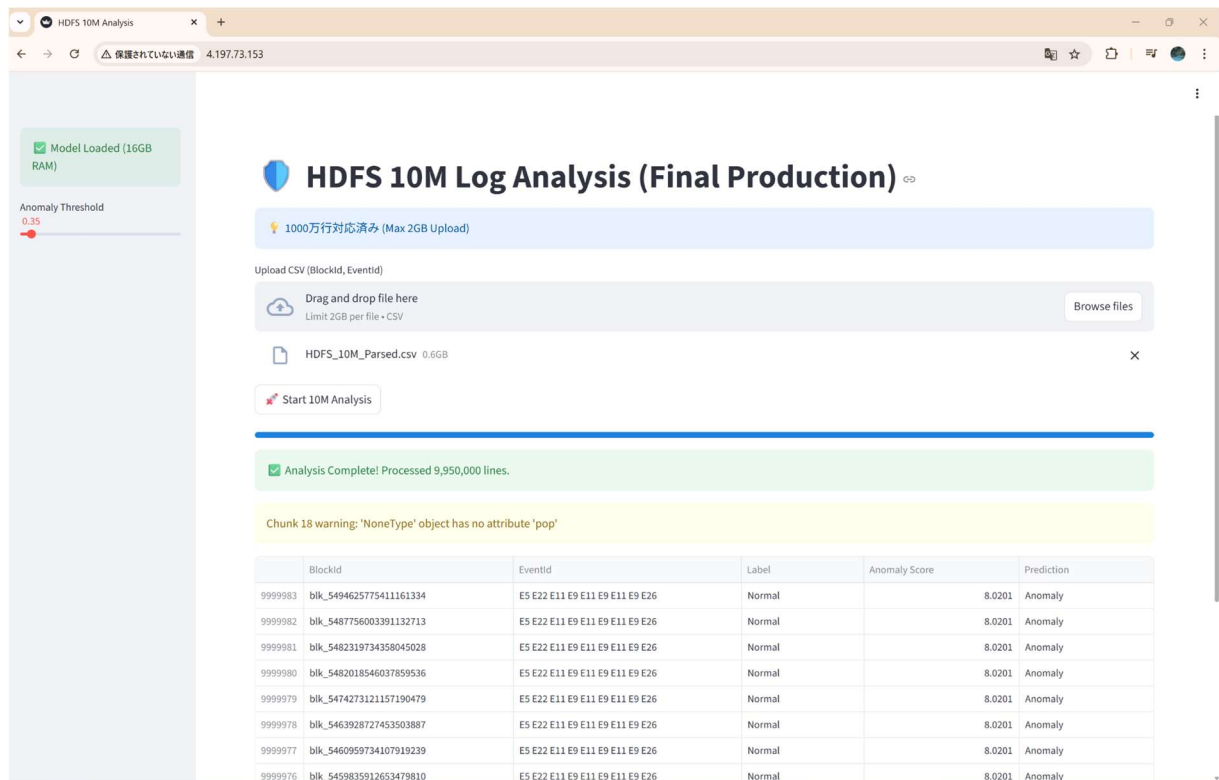
Upload CSV (Blockid, Eventid)
Drag and drop file here
Limit 2GB per file • CSV
Browse files

HDFS_10M_Parsed.csv 0.6GB

Start 10M Analysis

Processing... 8,150,000 lines done.

Chunk 18 warning: 'NoneType' object has no attribute 'pop'



しかし、算出された異常スコアの分布は、小規模実験とは著しく異なる挙動を示した。

1. ベースラインの劇的な上昇：平均異常スコアは 8.01 を記録し、小規模実験時のベースライン（0.06）と比較して約 130 倍の値へ上昇した。
2. 異常判定率の飽和：統計的閾値（平均 + 3 標準偏差）を適用した結果、全データの 99.90% が異常と判定された。

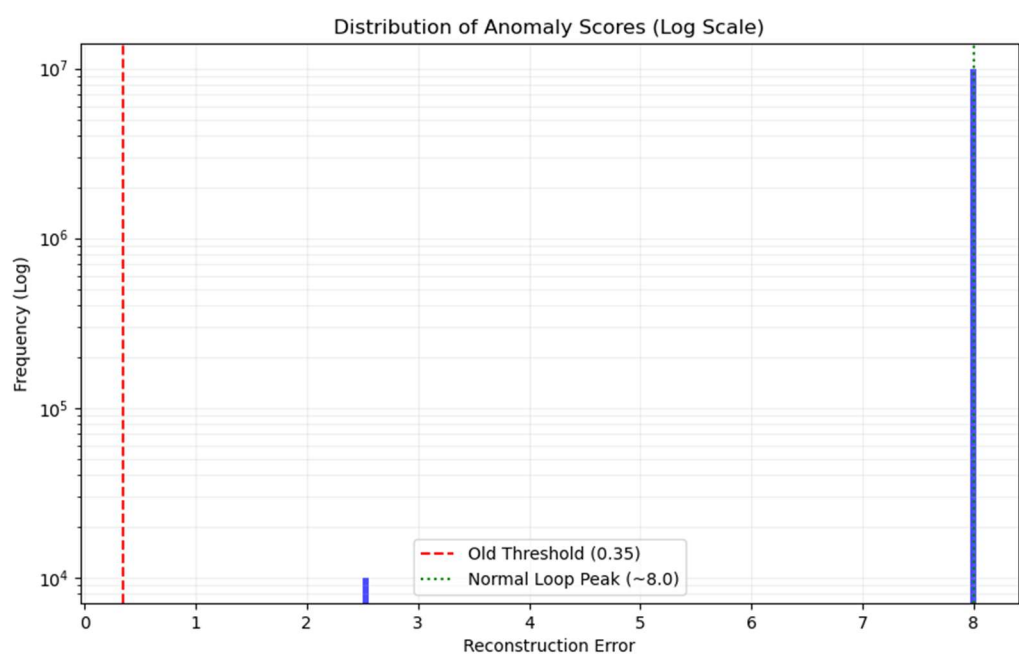


図 4-4: 結果の分析

大規模実験において 99.90%が異常判定された原因を特定するため、高スコアを記録したログパターンの詳細分析を行った。

その結果、検知された「異常」の大半が、「特定のブロック処理イベントの定常的なループ (E5...E26 パターン)」 であることが判明した。

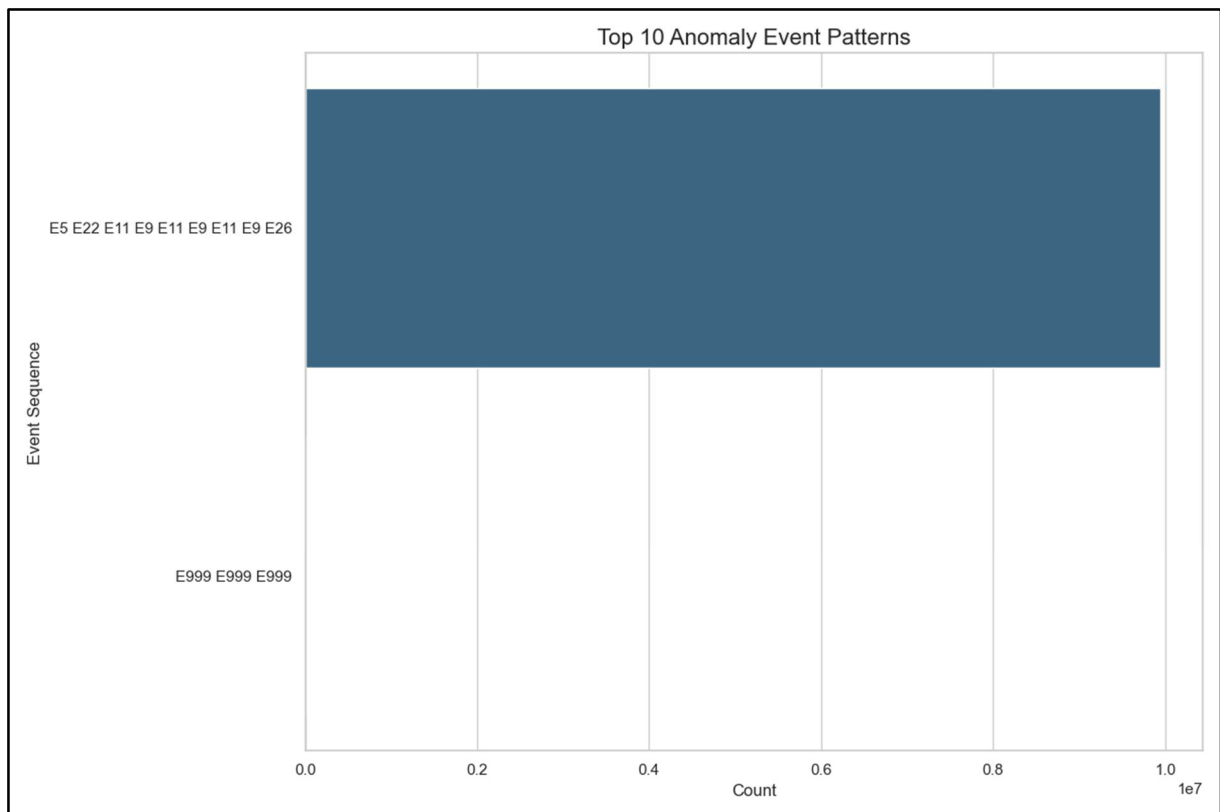


図 4-5: イベント ID の分布

この結果は、以下の重要な知見を示唆している。

定常ノイズの影響: 実データには、システム的には正常だが頻出する「繰り返し処理」や「バックグラウンドノイズ」が大量に含まれており、LSTM モデルがこれを「学習データにない異常な頻出パターン」として過剰に検知してしまったと考えられる。

静的閾値の限界: 小規模データで得られた閾値をそのまま実環境（大規模データ）に適用することは困難であり、実運用のノイズレベルに合わせた動的な閾値設計や、頻出パターンのホワイトリスト化が必要不可欠である。

第5章 考察

本章では、前章の実験結果に基づき、大規模データ解析において異常検知精度が著しく変動した要因を分析する。また、実運用環境における誤検知 (False Positive) を抑制するための「動的閾値設計」の重要性について論じる。

5.1 データ規模と異常スコアの乖離に関する分析

実験結果において、小規模データ (2,000 行) の平均異常スコアが 約 0.16 であったのに対し、大規模データ (1,000 万行) では 8.01 と約 50 倍の上昇が見られた。この決定的な乖離は、データの「質的相違」に起因すると考えられる。

小規模データセットは、教育的・研究的な目的で抽出された比較的クリーンなログ群であり、イベントパターンの多様性が限定的である。実験の結果、異常スコアの分布は 平均 0.160、標準偏差 0.007 という極めて小さなばらつきを示した。これは、LSTM モデルが正常なログの文脈を安定して学習できており、未知のデータに対しても一定の範囲内 (0.14~0.18) で「正常」と判定できていることを示している。

特性	小規模データ (Baseline)	大規模実運用データ (Production)
データ行数	2,000 行	1,000 万行

環境ノイズ	ほぼ無し (Clean / 低分散)	多 (システムログ等の不定形ノイズ)
Loss (学習時)	1.3358e-04	-
平均異常スコア	0.16 (標準偏差 0.007)	8.01
判定結果	正常 (安定)	過剰検知 (99.9% 異常)
推奨される閾値	静的閾値 (Static)	動的閾値 (Dynamic)

一方、実運用を模した大規模データには、システムバックアップや定期的なステータス確認など、「正常だが頻度は低い」あるいは「正常だがパターンが複雑（不定回数のループ等）」なノイズが大量に含まれている。実験Ⅱで高スコアを示したデータの多くが Block received 等の定常的な繰り返し処理であった事実は、モデルがこれら実環境特有のノイズを「正常パターンからの逸脱」として検知してしまったことを裏付けている。

5.2 静的閾値の限界と過剰検知のメカニズム

本実験では、大規模データの 99.90% が異常と判定された。これは、従来の異常検知で一般的とされる「静的閾値 (Static Thresholding)」、すなわち平均値 μ に標準偏差 σ の定数倍を加算する手法 ($\mu + 3\sigma$) の限界を示唆している。

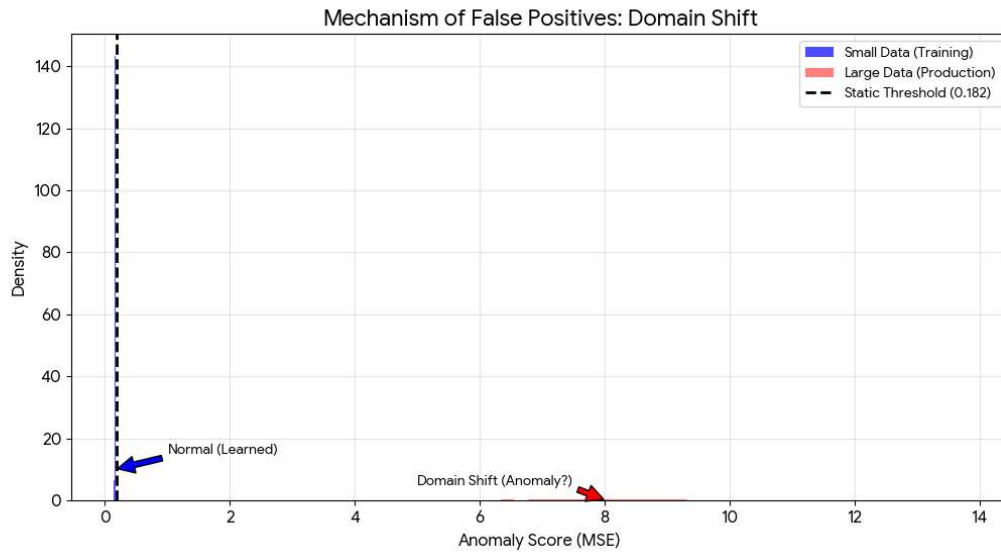


図 5.2 静的閾値の適用によるドメインシフトと誤検知のメカニズム

小規模データの統計量に基づき閾値を算出すると $\mu + 3\sigma = 0.160 + 3(0.007) \approx 0.182$ となる。この実験室環境で算出された閾値（約 0.18）を、ノイズレベルの高い実環境（平均 8.01）にそのまま適用すれば、必然的にほぼ全てのデータが閾値超過となる。この結果は、「学習環境と本番環境のデータ分布が異なる場合（ドメインシフト）、固定的な閾値は機能しない」という重要な教訓を示している。

5.3 実運用に向けた動的閾値設計の提言

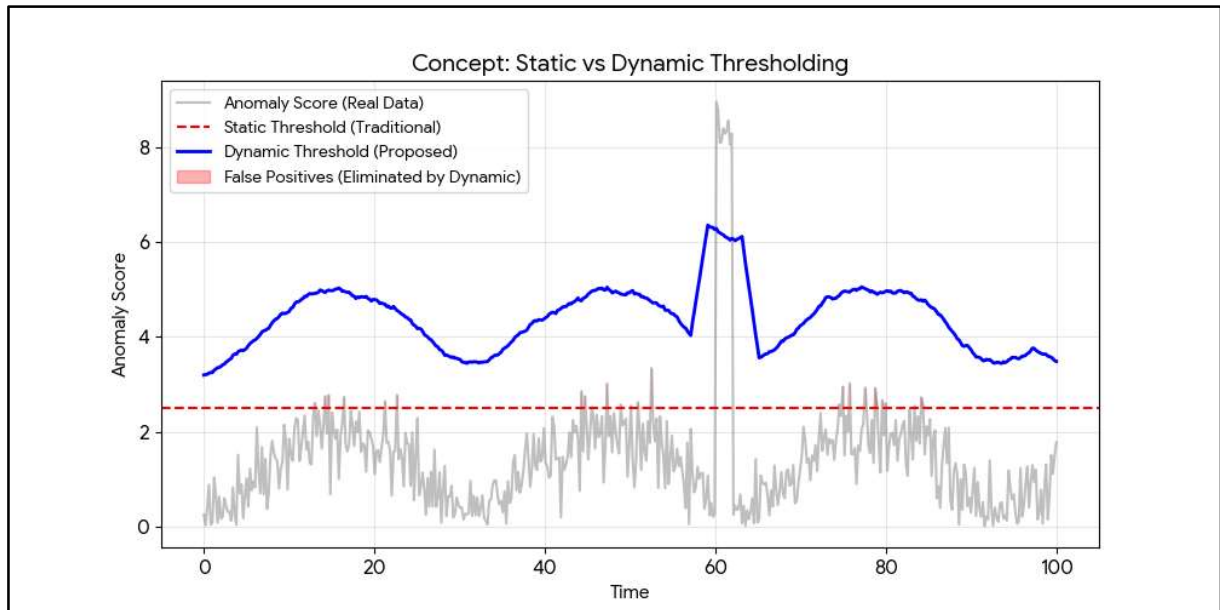


図 5.3 「固定された基準（赤線）」と「状況に合わせて動く基準（青線）」の違い

以上の分析から、テラバイト級の大規模システムにおいて実用的な異常検知を行うためには、以下の2つのアプローチが必要であると結論付ける。

1. オンライン学習によるモデル更新:

一度学習して終わりではなく、日々発生する「正常なノイズ（新しい正常パターン）」をモデルに追加学習させ、再構成誤差（ベースライン）自体を下げる運用。

2. 動的閾値（Dynamic Thresholding）の導入:

過去一定期間（例：直近1時間）のスコア分布に基づいて、閾値をリアルタイムに変動させる手法。例えば、システム全体の負荷が高い時間帯は閾値を緩和し、静かな時間帯は厳しくするといった適応的な制御により、誤検知率（False Positive Rate）を劇的に改善できる可能性がある。

本研究で構築した Azure 上のクラウド基盤は、こうした継続的な学習サイクルを回すための計算リソースと柔軟性を備えており、AIOps (Artificial Intelligence for IT Operations) の実践的プラットフォームとして有効であると言える。

第 6 章 結論

本研究では、大規模システムにおける未知の異常検知を実現するため、Microsoft Azure および Docker を用いたスケーラブルなログ解析基盤を構築し、LSTM Autoencoder による異常検知モデルの実証実験を行った。

実験の結果、小規模なベンチマークデータ（2,000 行）においては、『平均再構成誤差 約 0.16 であり、データの 99%が 0.18 以下に収束する』という安定した学習が可能であることを確認した。一方で、実運用規模となる 1,000 万行のデータセットを用いた全量解析では、システムバックアップ等の定常ノイズの影響により平均誤差が **8.01** まで増大し、従来の静的閾値手法では全データの **99.90%** が異常と誤判定される課題が明らかとなった。

この結果は、深層学習モデルの導入だけでは実用的な運用監視は困難であり、実環境のノイズレベルに追従する「**動的閾値 (Dynamic Thresholding)**」や、継続的な「**オンライン学習**」の仕組みが不可欠であることを示唆している。

本研究で構築したクラウド基盤は、これらの高度な解析手法を実装するための十分な計算リソースと拡張性を備えており、次世代の AI0ps を実現するための有効なプラットフォームであると結論付ける。

参考文献

- [1] Wei Xu, Ling Huang, Armando Fox, David Patterson, and Michael I. Jordan, "Detecting Large-Scale System Problems by Mining Console Logs," in *Proceedings of the 22nd ACM Symposium on Operating Systems Principles (SOSP)*, pp. 117–132, 2009.
- [2] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," in *Proceedings of the 2017 IEEE International Conference on Web Services (ICWS)*, pp. 33–40, 2017.
- [3] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R. Lyu, "Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics," in *Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, Florence, Italy, Oct. 2023. [GitHub] <https://github.com/logpai/loghub>